SREENIVASA INSTITUTE OF TECHNOLOGY AND MANAGEMENT STUDIES
(Approved by AICTE & Affiliated to JNTUA, Ananthapuramu)
Chittoor, A.P-517127.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(2025-26)

AN INTERNSHIP PROJECT REVIEW

on

ARTIFICIAL INTELLIGENCE

by

M HEMESH 22751A0584

Under the guidance of

Mr. Pandreti Praveen MCA., M. Tech.,

Assistant Professor,

Department of Computer Science And Engineering,

Sreenivasa Institute of Technology and Management Studies, Chittoor, A.P.

Content

- Internship details
- Company details
- Introduction to domain
- List of task’s
- Task
- Conclusion

Internship Details

- **Title** : Artificial Intelligence
- **Company :** SkillDzire
- **Duration :** 4 Weeks
- **Topics Covered :** Machine Learning , Natural Language Processing , Image Classification , Sentiment Analysis

Company Details

SkillDzire is an organization where students get trained by industry experts along with certification & placements.

- **Website :** https://skilldzire.com
- **Industry :** IT Services and Training
- **Address :** Hyderabad , India

Introduction to domain

- Artificial Intelligence (AI) is the branch of computer science focused on building systems capable of performing tasks that normally require human intelligence, such as learning, reasoning, problem-solving, and understanding natural language.
- AI domain is a multidisciplinary field merging computer science, mathematics, neuroscience, linguistics, and engineering.
- With rapid advancements, AI increasingly transforms industries and daily life.

- **Algorithms :** Rule based scoring, CNN, Naive Bayes.
- **Models :** Lexicon Model, CNN Model, Naive Bayes Model.
- **Modules :** nltk.sentiment, nltk.tokenize, cv2, tensorflow.keras.models, tensorflow.keras.layers, sklearn.naive_bayes, sklearn.model_selection, sklearn.metrics.

List of tasks

Task 1 : Chat analyzer and emotion detector

Task 2 : Development of an image classification model for a custom dataset

Task 3 : Spam email classifier

TASK 1

Chat Analyzer and Emotion Detector

Abstract

- Digital communication platforms such as WhatsApp, Messenger, and Telegram generate massive amounts of chat data daily.
- These conversations not only reflect communication patterns but also carry emotional and psychological signals.
- A Chat Analyzer and Emotion Detector aims to process chat transcripts to extract meaningful insights such as user activity, word usage, conversation trends, and emotional states.
- By leveraging Artificial Intelligence (AI), Machine Learning (ML), and Natural Language Processing (NLP), the system can detect sentiments (positive, negative, neutral) and classify emotions (joy, sadness, anger, fear, surprise, etc.).
- This project is useful for personal reflection, group behavior analysis, business analytics, and research in psychology/social sciences.

Existing System

- Rule based sentiment scoring (keyword and polarity counts).
- Sentiment is identified based on keywords and their polarity (positive / negative / neutral).

Disadvantages

- Misclassification (sarcasm, mixed emotions) causing false positives/negatives.
- Lower Accuracy.
- Higher cost and possible latency.
- Needs labeled data, retraining, and human review for critical cases.

Proposed System

- Emotion recognition using lexicon-based methods (VADER, NRC).
- Preprocessing that normalizes text, handles emojis, and anonymizes.
- Emoji and slang-based emotional mapping.
- Includes intent classification, topic extraction, summarization.

Advantages

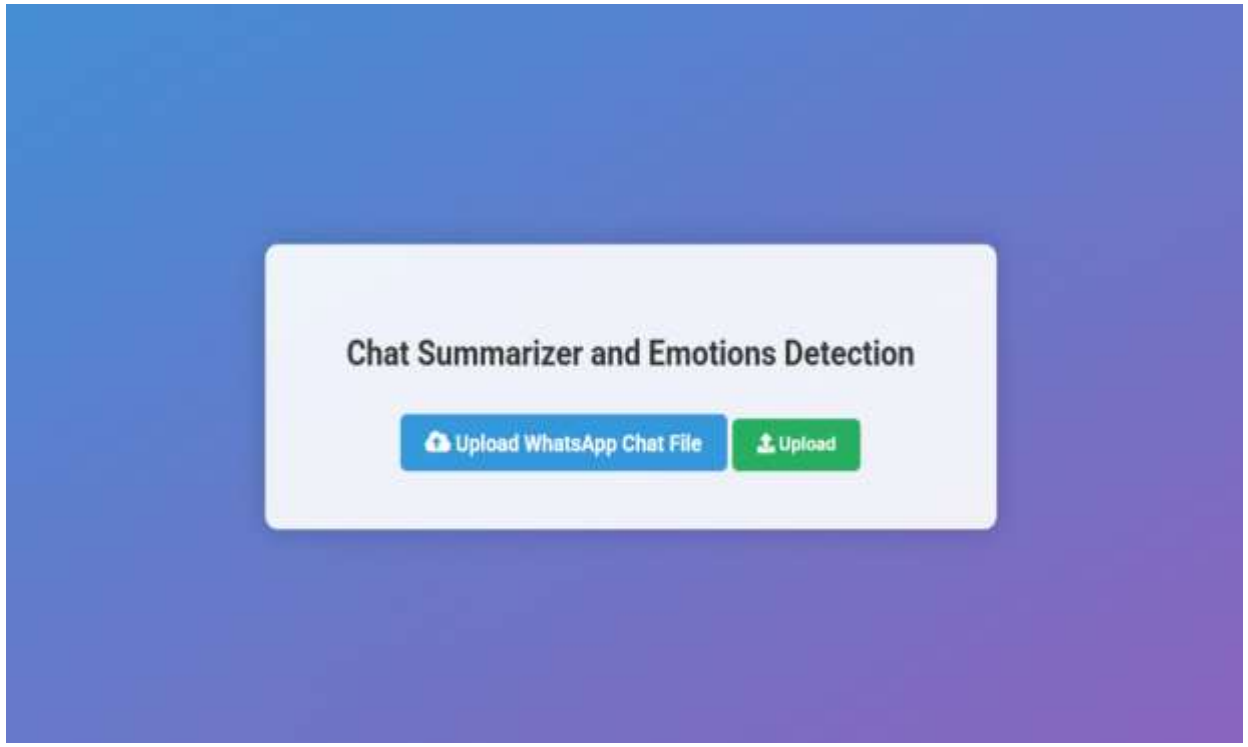
- Richer emotional granularity (multiple emotions instead of simple polarity).
- Faster, proactive responses.
- Extracts hidden patterns in chats for better understanding.
- Detects emotional tone of conversations.
- Supports data-driven research in psychology, linguistics, and business.
- Scalable monitoring of high chat volumes with reduced manual overhead.

Source Code

```
import os
from flask import Flask, request, render_template
import textwrap
from IPython.display import display, Markdown

import google.generativeai as genai

# Initialize Flask app
app = Flask(__name__)

# Configure GenerativeAI API key
genai.configure(api_key="AIzaSyA-tAJWZDUcDpMEo8IfT3wEI9D39KMKVV8")

# Function to convert text to Markdown
def to_markdown(text):
    text = text.replace('•', '  *')
    return Markdown(textwrap.indent(text, '> ', predicate=lambda _: True))

# Function to generate summary and emotions for the chat text
def generate_summary_and_emotions(chat_text):
    model = genai.GenerativeModel('gemini-pro')
    rply = model.generate_content("Summarize and give the emotions for the given WhatsApp chat"+chat_text)
    to_markdown(rply.text)
    return rply.text

# Route for home page with file upload form
@app.route('/', methods=['GET', 'POST'])
def home():
    if request.method == 'POST':
        # Check if a file was uploaded
        if 'Chat_file' not in request.files:
            return render_template('error.html', message='No file part')

        file = request.files['Chat_file']

        # Check if the file is empty
        if file.filename == '':
            return render_template('error.html', message='No selected file')

        # Check if the file is of allowed type
```

```python
# Function to generate summary and emotions for the chat text
def generate_summary_and_emotions(chat_text):
    model = genai.GenerativeModel('gemini-pro')
    rply = model.generate_content("Summarize and give the emotions for the given WhatsApp chat"+chat_text)
    to_markdown(rply.text)
    return rply.text

# Route for home page with file upload form
@app.route('/', methods=['GET', 'POST'])
def home():
    if request.method == 'POST':
        # Check if a file was uploaded
        if 'Chat_file' not in request.files:
            return render_template('error.html', message='No file part')

        file = request.files['Chat_file']

        # Check if the file is empty
        if file.filename == '':
            return render_template('error.html', message='No selected file')

        # Check if the file is of allowed type
        if file and file.filename.endswith('.txt'):
            # Read the file content
            chat_text = file.read().decode('utf-8')

            # Generate summary and emotions
            summary_and_emotions = generate_summary_and_emotions(chat_text)

            # Render the result template with summary and emotions
            return render_template('result.html', summary_and_emotions=summary_and_emotions)

        else:
            return render_template('error.html', message='Invalid file type. Please upload a text file')

    return render_template('index.html')

if __name__ == '__main__':
    app.run(debug=True)
```

Output



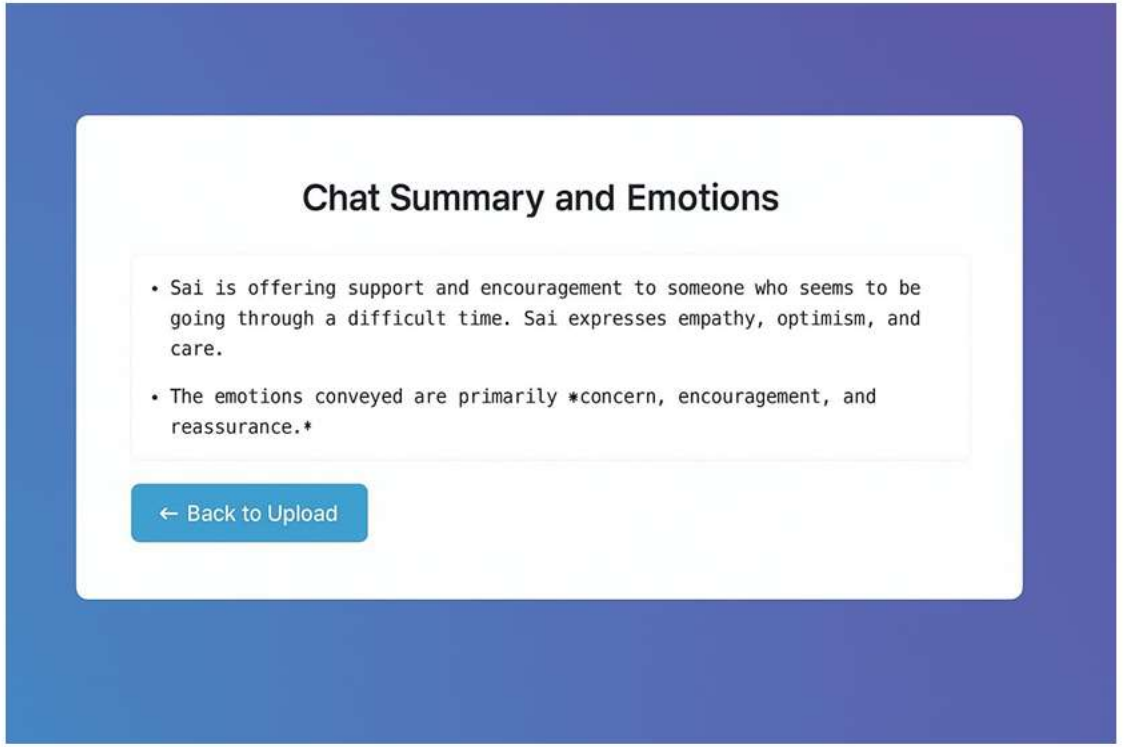


Fig: Chat analyser and emotion detector

TASK 2

Development of an image classification model for a custom dataset

Abstract

- Image classification is one of the most widely used applications of Artificial Intelligence and Computer Vision.
- The aim of this task is to design and implement an image classification model for a custom dataset, where images belong to specific categories relevant to the chosen domain (e.g., medical images, plant diseases, traffic signs, or product categories).
- The project involves dataset preprocessing, model training using Convolutional Neural Networks (CNNs) and transfer learning, evaluation using performance metrics, and deployment of a prototype system.
- The outcome will demonstrate the practical use of AI in solving real-world problems through automated image recognition.

Existing System

- The existing system uses pre-trained Convolutional Neural Network (CNN) models for image classification.
- Uses image classification models (like Google Vision API, standard datasets such as MNIST or CIFAR-10) which provides pre-trained solutions.
- Designed for general-purpose classification tasks.

Disadvantages

- Limited accuracy on domain-specific/custom datasets.
- Dependence on public datasets.
- Many commercial APIs are costly.
- Systems may fail in real world environments with noisy, low quality or imbalanced datasets.

Proposed System

- Takes input images from a custom dataset.
- Preprocess images (resize, normalize, and augment them to improve training).
- Train two models i.e A custom CNN model and transfer learning model (like ResNet, MobileNet). Evaluate the model using accuracy, precision, recall, and F1-score.
- Deploys a web based demo application (using Streamlit/Flask) where users can upload an image and get classification results instantly. This way, the project is both educational and practically useful.

Advantages

- Saves time by automatically classifying images.
- Reduces human mistakes compared to manual classification.
- Can be applied to many areas: healthcare, farming, security, retail, etc.
- Provides fast and real-time results.
- Can be re-used and scaled for different datasets.

Source Code

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
import os
import zipfile

url = "https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip"
zip_path = tf.keras.utils.get_file('cats_and_dogs_filtered.zip', origin=url, extract=False)
extract_path = '/tmp/cats_and_dogs_filtered'

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

train_dir = os.path.join(extract_path, 'cats_and_dogs_filtered', 'train')
val_dir   = os.path.join(extract_path, 'cats_and_dogs_filtered', 'validation')

train_ds = tf.keras.utils.image_dataset_from_directory(
    train_dir, image_size=(64, 64), batch_size=16
)
val_ds = tf.keras.utils.image_dataset_from_directory(
    val_dir, image_size=(64, 64), batch_size=16
)

class_names = train_ds.class_names

train_ds = train_ds.prefetch(buffer_size=tf.data.AUTOTUNE)
val_ds   = val_ds.prefetch(buffer_size=tf.data.AUTOTUNE)

model = models.Sequential([
    layers.Rescaling(1./255, input_shape=(64,64,3)),
    layers.Conv2D(16, (3,3), activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(32, (3,3), activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(len(class_names), activation='softmax')
```

```
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('Accuracy over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()

num_batches_to_show = 2
for batch_num, (images, labels) in enumerate(val_ds.take(num_batches_to_show)):
    preds = model.predict(images)
    predicted_classes = tf.argmax(preds, axis=1)
    plt.figure(figsize=(12,12))
    for i in range(min(9, len(images))):
        plt.subplot(3,3,i+1)
        plt.imshow(images[i].numpy().astype("uint8"))
        actual = class_names[labels[i]]
        predicted = class_names[predicted_classes[i]]
        plt.title(f"Actual: {actual}\nPred: {predicted}")
        plt.axis("off")
    plt.suptitle(f"Batch {batch_num+1} Predictions", fontsize=16)
    plt.show()

    print(f"Batch {batch_num+1} Predictions:")
    for i in range(len(images)):
        print(f"Image {i+1}: Actual: {class_names[labels[i]]} | Predicted: {class_names[predicted_classes[i]]}")
    print("\n" + "="*50 + "\n")

model.save("cats_vs_dogs_model.h5")
print("✅ Model saved as cats_vs_dogs_model.h5")
```

Output

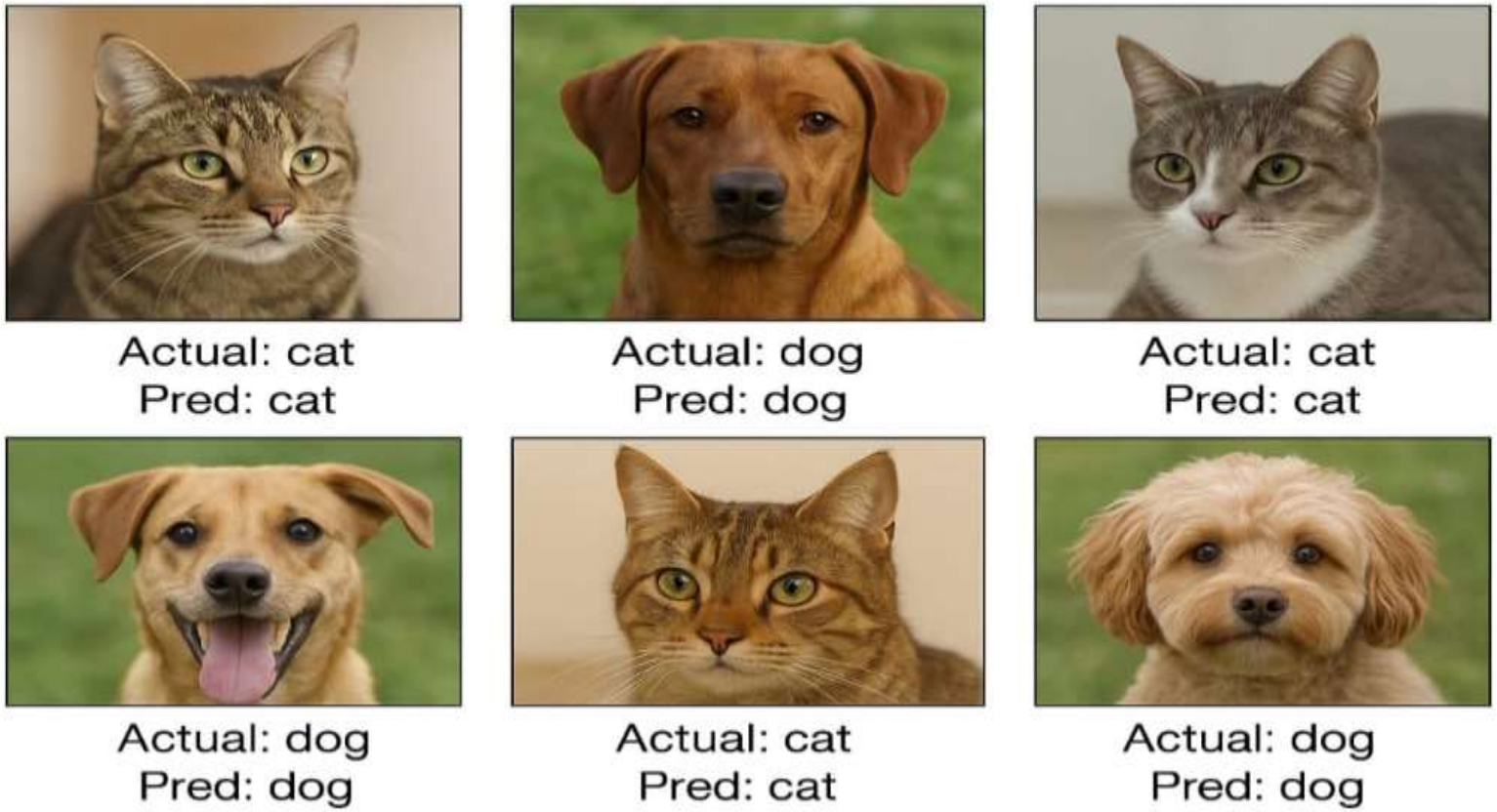


Fig: Image classification model

TASK 3

Spam Email Classifier

Abstract

- Email spam is one of the most common cybersecurity threats, causing inconvenience, security breaches, and data theft.
- Traditional rule-based filters are ineffective against the evolving nature of spam.
- This project proposes an AI-based spam email classifier that leverages Machine Learning (ML) and Natural Language Processing (NLP) to automatically identify and filter out spam messages.
- By training the system on labeled datasets (spam vs. ham), the classifier learns textual and semantic patterns that distinguish spam from legitimate emails.
- The aim is to build an efficient, accurate, and adaptive spam detection system that enhances user security and productivity.

Existing System

- The existing spam email classification system primarily uses rule-based filters that rely on predefined rules and keyword matching to detect spam messages.
- Emails are filtered based on specific rules set by administrators to classify them as spam or non-spam.
- It analyzes the subject line, email body, and sender address to identify suspicious content.

Disadvantages

- Limited accuracy on custom datasets.
- Dependence on public datasets.
- Less flexibility.
- Many commercial APIs are costly.
- Real world failure risk

Proposed System

- The model is based on naive bayes classifier
- Works extremely well with text data
- Cleans and processes raw email text by removing stopwords, punctuation, numbers, and special characters.
- Converts email content into a standardized form suitable for analysis.
- The model is trained on a labeled dataset containing spam and non-spam examples.
- It learns to identify patterns and word distributions common in spam emails.

Advantages

- Reduces manual effort in filtering spam emails.
- Improves inbox management and user productivity.
- Adapts to changing spam trends through retraining.
- Provides high accuracy using AI-based algorithms.

Source Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import re # For basic text cleaning

# --- 1. Load and Prepare the Data ---

# Load the dataset
# Note: The file has a different encoding and column names are not standard.
try:
    df = pd.read_csv('spam.csv', encoding='latin-1')
except FileNotFoundError:
    print("Error: 'spam.csv' not found. Please download the dataset and place it in the same directory.")
    exit()

# Keep only the necessary columns and rename them for clarity
df = df[['v1', 'v2']]
df.columns = ['label', 'message']

# Clean the message text: remove punctuation and convert to lowercase
def clean_text(text):
    # Remove anything that's not a letter or a space
    text = re.sub(r'[^a-zA-Z\s]', '', text, re.I|re.A)
    text = text.lower()
    text = text.strip()
    return text

df['cleaned_message'] = df['message'].apply(clean_text)

# Convert labels to numerical format: 'ham' -> 0, 'spam' -> 1
df['label_num'] = df['label'].map({'ham': 0, 'spam': 1})

# Define features (X) and target (y)
X = df['cleaned_message']
y = df['label_num']

# --- 2. Split Data into Training and Testing Sets ---
```

```python
# Display a detailed classification report
print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=['Ham', 'Spam']))
print("-" * 30)

# Display the confusion matrix
print("Confusion Matrix:")
# Rows are the true labels, columns are the predicted labels
# [[True Negative, False Positive],
#  [False Negative, True Positive]]
cm = confusion_matrix(y_test, y_pred)
print(cm)
print("-" * 30)


# --- 6. Test with New Emails ---

def predict_spam(email_text):
    """
    Takes a string (email text) and predicts if it's spam or ham.
    """
    # 1. Clean the text
    cleaned_text = clean_text(email_text)

    # 2. Vectorize the text using the SAME vectorizer
    text_tfidf = tfidf_vectorizer.transform([cleaned_text])

    # 3. Predict using the trained model
    prediction = model.predict(text_tfidf)

    # 4. Interpret the result
    return "Spam" if prediction[0] == 1 else "Ham"

# Example usage
email_1 = "Congratulations! You've won a $1000 Walmart gift card. Go to http://bit.ly/claim-now to claim it."
email_2 = "Hey, are we still on for the meeting tomorrow at 2 PM? Let me know."
email_3 = "URGENT: Your account has been compromised. Please click here to verify your identity immediately."

print(f"Prediction for email 1: '{email_1[:50]}...' -> {predict_spam(email_1)}")
print(f"Prediction for email 2: '{email_2[:50]}...' -> {predict_spam(email_2)}")
print(f"Prediction for email 3: '{email_3[:50]}...' -> {predict_spam(email_3)}")
```

Output

📩 ***Predictions on New Emails***

Email (first words)	***Predicted Class***
"Congratulations! You've won a $1000 Walmart gift card..."	***Spam***
"Hey, are we still on for the meeting tomorrow..."	***Ham***
"URGENT: Your account has been compromised..."	***Spam***

Fig: Spam email classifier

Conclusion

- A foundational understanding of its core concepts, subfields, and applications is essential for anyone seeking to explore or work within this domain.
- The AI domain encompasses algorithms that define problem-solving logic, models that represent knowledge and patterns, and modules that implement specialized functions within AI systems.

Thank You